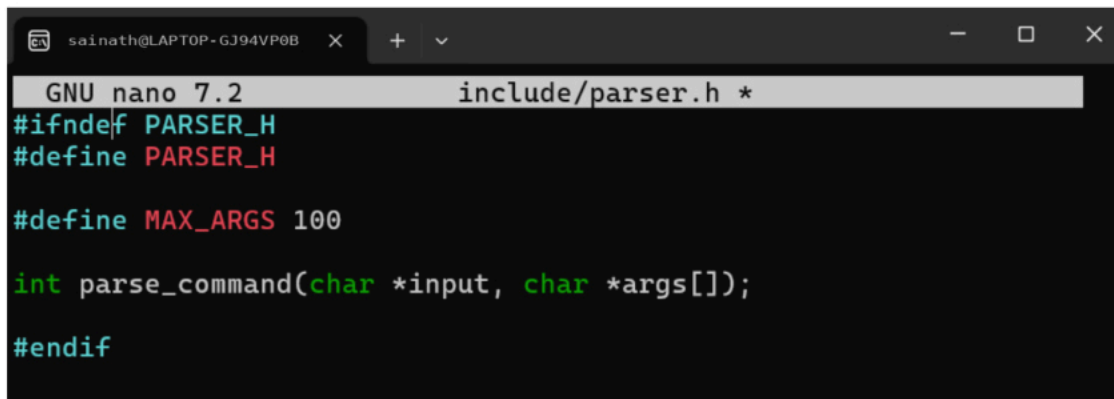


Skill Week 5

1 . To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Aim : To implement double quotes in a shell parser

Step 1 — Create parser.h



```
GNU nano 7.2 include/parser.h *
#ifndef PARSER_H
#define PARSER_H

#define MAX_ARGS 100

int parse_command(char *input, char *args[]);

#endif
```

Step 2 — Create parser.c



```
GNU nano 7.2 src/parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "parser.h"

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int in_quotes = 0;
    int i = 0;
    int j = 0;

    char token[1024];

    while (input[i] != '\0')
    {
        /* Skip spaces outside quotes */
        while (isspace((unsigned char)input[i]) && !in_quotes)
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        j = 0;

        /* Read one token */
        while (input[i] != '\0')
        {
            /* Handle double quotes */
            if (input[i] == '"')
            {
                in_quotes = !in_quotes;
                i++;
                continue;
            }
        }
    }
}
```

```

/* Space ends token only outside quotes */
if (isspace((unsigned char)input[i]) && !in_quotes)
{
    break;
}

/* Variable expansion */
if (input[i] == '$')
{
    char variable[100];
    int k = 0;

    i++;

    while (isalnum((unsigned char)input[i]) ||
           input[i] == '_')
    {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0; value[x] != '\0'; x++)
        {
            token[j++] = value[x];
        }

        continue;
    }

    token[j++] = input[i++];
}

```

```

/* Check for unmatched double quote */
if (in_quotes)
{
    printf("Error: unmatched double quote\n");
    return -1;
}

token[j] = '\0';

if (j > 0)
{
    args[argc] = malloc(strlen(token) + 1);

    if (args[argc] == NULL)
    {
        printf("Memory allocation failed\n");
        return -1;
    }

    strcpy(args[argc], token);
    argc++;

    if (argc >= MAX_ARGS - 1)
    {
        break;
    }
}

/* Move past the space */
if (input[i] != '\0')
{
    i++;
}

args[argc] = NULL;

return argc;
}

```

```

/* Check for unmatched double quote */
if (in_quotes)
{
    printf("Error: unmatched double quote\n");
    return -1;
}

token[j] = '\0';

if (j > 0)
{
    args[argc] = malloc(strlen(token) + 1);

    if (args[argc] == NULL)
    {
        printf("Memory allocation failed\n");
        return -1;
    }

    strcpy(args[argc], token);
    argc++;

    if (argc >= MAX_ARGS - 1)
    {
        break;
    }
}

/* Move past the space */
if (input[i] != '\0')
{
    i++;
}

args[argc] = NULL;

return argc;
}

```

 Snipping Tool

Screenshot copied to clipboard
Automatically saved to snipping tool

Step 3 — Create a test program

```
GNU nano 7.2 src/main.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "parser.h"

int main()
{
    while (1)
    {
        char input[1024];
        char *args[MAX_ARGS];

        printf("ossp> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nExiting shell...\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        int argc = parse_command(input, args);

        if (argc == -1)
        {
            continue;
        }

        printf("Arguments:\n");

        for (int i = 0; i < argc; i++)
        {
            printf("args[%d] = [%s]\n", i, args[i]);
            free(args[i]);
        }

        return 0;
    }
}
```

```
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano src/parser.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano include/parser.h
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano src/shell.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano include/parser.h
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano src/parser.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano src/main.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ gcc src/main.c src/parser.c -Iinclude -o quote_shell
sainath@LAPTOP-GJ94VP0B:~/OSSP$ ./quote_shell
ossp> echo "Hello World"
Arguments:
args[0] = [echo]
args[1] = [Hello World]
ossp>
```

Step 4— Test variable expansion

```
ossp> echo "Hello $USER"
Arguments:
args[0] = [echo]
args[1] = [Hello sainath]
ossp>
```

Step 5 — Test mixed/nested tokens

```
ossp> echo "Hello World" Welcome
Arguments:
args[0] = [echo]
args[1] = [Hello World]
args[2] = [Welcome]
ossp> |
```

Step 6 — Test unmatched double quotes

```
ossp> echo "Hello World
Error: unmatched double quote
ossp>
```

Step 7— Final quoted-command tests

```
ossp> echo "Hello" "World"
Arguments:
args[0] = [echo]
args[1] = [Hello]
args[2] = [World]
ossp> echo "User: $USER"
Arguments:
args[0] = [echo]
args[1] = [User: sainath]
ossp> echo "Welcome to $USER system"
Arguments:
args[0] = [echo]
args[1] = [Welcome to sainath system]
ossp> |
```

Observation:

The shell parser was successfully implemented to support double quotes. Spaces within double quotes were preserved, quoted text was treated as a single argument, and environment variables such as \$USER were expanded correctly. The parser also handled quoted and unquoted tokens together and detected unmatched double quotes. The quoted commands were tested successfully.

Result:

Thus, double-quote handling was successfully implemented in the shell parser, and the required quoted-command tests were executed successfully.